Powershell Administration

Working with OUs

Creating OUs with New-ADOrganizationalUnit

- New-ADOrganizationalUnit
  [-name <ou>]
  [-Path <dn>]
  [-description <description>]
- Get-ADOrganizationalUnit
  [-Identity] <ADOrganizationalUnit>
  - Get-ADOrganizationalUnit -identity “**<DN>**”
- Get-ADOrganizationalUnit
  [-Filter] <ADOrganizationalUnit>
  - Get-ADOrganizationalUnit -Filter ‘Name –like “<name>”’

Do this…

- Create an OU called “**Mod7PWSH**” in the root of your domain. Give the OU a description of “**Puns and Things**”.
- Then, display that OU object on the screen.

Working with Users

Displaying User Information with `Get-ADUser`

- `Get-ADUser [-Identity] <ADUser>`
- `Get-ADUser [-Identity] <ADUser> –Properties [property1, property2,…]`
- You can identify a user by its distinguished name (**DN**), **GUID**, security identifier (**SID**), Security Accounts Manager (**SAM**) account name or name

Example:

- `Get-ADUser dmite`
- `Get-Aduser dmite -Properties *`

Creating Users with New-ADUSER

- New-ADUser
  [-name <cnName>]
  [-Path <ouDN>]
  [-SamAccountName <name>]
  [Options]
- Get-Aduser
  [-Filter <string>]
- Get-Aduser
  [-Identity <Aduser>]

New-ADUser Switches	
-AccountExpirationDate 12/31/2012	-NewPassword (ConvertTo-SecureString “password" -AsPlainText -Force)
-CannotChangePassword 1 (true)	-ChangePasswordAtLogon 1 (true) Note this one
-City “Moncton” Note this one	-Company “NBCC”
-Country CA	-Department “Networking”
-Description “Net Student”	-DisplayName “Johnny Smith”
-Division “Students”	-EmailAddress “jsmith@johnnyworld.com”
-Enabled 1 (true)	-GivenName Johnny
-HomeDirectory \\server01\home\jsmith	-HomeDrive Z:
-HomePage www.johnnyworld.com	-HomePhone 506-666-5555
-LogonWorkstations PC10 (separate names by commas, use SamAccountName or FQDN)	-Manager JDoe (either the SamAccountName or DN of the user)
-Office “Room 202”	-PasswordNeverExpires 1 (true)
-PasswordNotRequired 1 (true)	-ProfilePath “\\server1\profiles\jsmith”
-ScriptPath “logon.cmd” (Name of logon script)	-StreetAddress “123 Smith Street”
-Surname Smith	-Title Student
-UserPrincipalName jsmith@nbcc.local	

Do this…

- Create a *password-less* user
  - with a name of “**Chris P Bacon**”
  - in the **MOD7PWSH** organizational unit
  - with a samAccount name of “cbacon”
- Display **Chris P Bacon** to the console.

Do this…

- Create a user
  - with a name of “**Dinah Mite**”
  - in the **MOD7PWSH** organizational unit
  - with a Pre-Windows Logon name of “**dmite**”
  - a surname of **Mite**
  - a given name of **Dinah**
  - a UPN of **dmite@<yourdomain>.local**
  - and a Password of “**1234Qwert**”.
- Display Dinnah Mite to the console.

Deleting Users using Remove-ADUser

- `Remove-ADUser`
  `[-identity <adUser>]`
  `[-Path <ouDN>]`
  `[-SamAccountName <name>]`

- You can identify a user by its distinguished name (DN), GUID, security identifier (SID) or security accounts manager (SAM) account name
  - Don't' know any of those? `Get-Aduser <samid>`

Do this…

- Remove the user **cbacon**.
- Re-create Cbacon.

Modifying Users with PowerShell

- To set properties for an existing user, use the `Set-ADUser` cmdlet
  - Uses the **same parameters and syntax** as `New-ADUser`

```
Set-ADUser <samid> -Manager <samid>
```

- To disable an existing user account, use the `Disable-ADAccount` cmdlet

```
Disable-ADAccount <samid>
```

- To re-enable a user account, use the `Enable-ADAccount` cmdlet

```
Enable-ADAccount <samid>
```

- To unlock a locked-out account, use the `Unlock-ADAccount` cmdlet

```
Unlock-ADAccount <samid>
```

New-ADUser Switches	
-AccountExpirationDate 12/31/2012	-NewPassword (ConvertTo-SecureString “password" -AsPlainText -Force)
-CannotChangePassword 1 (true)	-ChangePasswordAtLogon 1 (true)
-City “Moncton”	-Company “NBCC”
-Country CA	-Department “Networking”
-Description “Net Student”	-DisplayName “Johnny Smith”
-Division “Students”	-EmailAddress “jsmith@johnnyworld.com”
-Enabled 1 (true)	-GivenName Johnny
-HomeDirectory \\server01\home\jsmith	-HomeDrive Z:
-HomePage www.johnnyworld.com	-HomePhone 506-666-5555
-LogonWorkstations PC10 (separate names by commas, use SamAccountName or FQDN)	-Manager JDoe (either the SamAccountName or DN of the user)
-Office “Room 202”	-PasswordNeverExpires 1 (true)
-PasswordNotRequired 1 (true)	-ProfilePath “\\server1\profiles\jsmith”
-ScriptPath “logon.cmd” (Name of logon script)	-StreetAddress “123 Smith Street”
-Surname Smith	-Title Student
-UserPrincipalName jsmith@nbcc.local	

Do this…

- Set **smattis** as dmite’s **Manager**
- **Disable** cbacon’s account
- Set the **password** of Dmites account to **Pa$$w0rd**.
- Unlock dmite’s account and then check if it is unlocked.

Working with Groups

Displaying Group Information with `Get-ADGroup`

- To view information about an existing group, use the `Get-ADGroup` cmdlet
- `Get-ADGroup`
  `[Identity <ADGroup>]`
- `Get-ADGroup`
  `[-Filter <string>]`
- Examples
  - `Get-ADGroup –Identity *`
  - `Get-ADGroup –Identity "PunnyGroup"`
  - `Get-ADGroup -Filter "GroupCategory -eq 'Security' -and GroupScope -eq 'Global'"`

Get-ADGroupMember

- To view the mmbers of a group
- `Get-ADGroupMember`
  `[Identity <ADGroup>]`

- Examples:
  - `Get-ADGroupMember –Identity *`
  - `Get-ADGroupMember –Identity “Domain Users”`

Creating Groups Using New-ADGroup

- New-ADGroup
  [-name <groupname>]
  [-GroupScope d|g|u]
  [-GroupCategory s|d]
  [-path <Oudn>]

GroupScope can use:

```
d / domainlocal / 0
g / global / 1
u / universal / 2
```

Group Category can use

```
D / distribution / 0
S / security / 1
```

Do this…

- Create a
  - **global security group**
  - called “**PunnyGroup”**
  - In the **MOD7PWSH** OU.
- Display the group to the screen.

Modifying Groups with Powershell

- To set properties of an existing group, use the `Set-ADGroup` cmdlet
  - `Set-ADGroup –Identity <groupDN> -Description “<string>" -GroupScope 0/1/2 GroupCategory 0/1`
- To add a user to an existing AD group, use the cmdlet `Add-ADGroupMember`
  - `Add-ADGroupMember <groupDN> -Members <samID1>,<samID2>`
- To Remove a group member, use the cmdlet `Remove-ADGroupMember`
  - `Remove-ADGroupMember <groupDN> –Members <samID1>,<samID2>`

Do this…

- Set the **Description** of “**Punny Groups**” to “**Whatever**” and change the group to a **Universal Distribution Group**.
- Add cbacon and dmite as members.
- Display the group.

Deleting Groups with Remove-ADGroup

- To delete a group, use the **Remove-ADGroup** cmdlet

```
Remove-ADGroup –Identity <groupName>
```

Bulk User Creation

User Creation Automation

Bulk User Creation

- Users can be created in bulk in several different ways. They all require one of the looping mechanisms found in PWSH.
  - `For`, `ForEach`, `ForEach-Object`, `Do` (`while`/`until`).
- Your input mechanism can differ as well.
  - Using an Array or hash table as input for the required properties of those accounts. This could also be a simple text file.
  - Using a common exchange format (CSV, XML) as the input source.
  - Using an integrative, incrementing loop

This is not an exhaustive list.

Creating users using an Array.

- Step 1: Define an array of property names (usually the name of the account).
  - Because arrays only have one item in them, you will need multiple arrays if you want to specify multiple properties.

```
1   $userArray = "Dan","Mike","Suze","Lynn"
```

- Step 2: Using a `ForEach` or a `ForEach-Object` loop, iterate through the array one object at a time.

```
3 ForEach ($user in $userArray){
4
5     New-ADUser -Name $user -PassThru | Select-Object Name
6
7 }
```

Creating users using an Array File

- Step 1: Define an array of property names (usually the name of the account) and store those items in a text file.
  - Using Get-Content, initialize a variable with the Names in the text file.

```
$userArray = (Get-Content C:\pwshWork\MSII\userArray.txt)
```

- Step 2: Using a `ForEach` or a `ForEach-Object` loop, iterate through the array one object at a time.

```
ForEach ($user in $userArray){

    New-ADUser -Name $user -PassThru | Select-Object Name

}
```

A note about input Method.

- Now that you've seen arrays, file arrays, and forEach, you can substitute a different exchange format in place of the array.
- The type of exchange format you use is less important that *what's in it*. Arrays have only one object in them, so they are sub-optimal for this exercise.
- What we really want to use is a *collection*. Collections hold many objects and their properties. We can configure many properties for many accounts using:
  - CSVs
  - XMLs
  - Collections stored in memory (the result of some other command being piped)

Using a collection as input.

- CSVs are the most common for of exchange format. They represent a collection (table).
- A CSV is a Comma Separated Value file. It is a text representation of a table.
  - The first row are the headers of the table.
  - Each column is denoted by a comma
  - Each line is a new object.

```
name,givenName,surname,Password
will,will,williams,Pa$$w0rd
wanda,wanda,walker,Pa$$w0rd
whitney,whitney,white,Pa$$w0rd
westley,westley,weber,Pa$$w0rd
```

=

name	givenName	surname	Password
will	will	williams	Pa$$w0rd
wanda	wanda	walker	Pa$$w0rd
whitney	whitney	white	Pa$$w0rd
westley	westley	weber	Pa$$w0rd

Using a collection as input.

```
name    givenName surname  Password
----    --------- -------  --------
will    will      williams Pa$$w0rd
wanda   wanda     walker   Pa$$w0rd
whitney whitney   white    Pa$$w0rd
westley westley   weber    Pa$$w0rd
```

- It doesn't matter what type of collection we've used. There are a few things that we should note:
  - Each column header is a property of the collection
  - That property can be reference in any of the usual ways

```
PS C:\pwshwork\MSII> (ipcsv .\userNames.csv).name
will
wanda
whitney
westley
```

```
PS C:\pwshwork\MSII> Ipcsv .\userNames.csv | Select Name,Password

name    Password
----    --------
will    Pa$$w0rd
wanda   Pa$$w0rd
whitney Pa$$w0rd
westley Pa$$w0rd
```

```
PS C:\pwshwork\MSII> Ipcsv .\userNames.csv | GM

   TypeName: System.Management.Automation.PSCust

Name        MemberType   Definition
----        ----------   ----------
Equals      Method       bool Equals(System.Obje
GetHashCode Method       int GetHashCode()
GetType     Method       type GetType()
ToString    Method       string ToString()
givenName   NoteProperty string givenName=will
name        NoteProperty string name=will
Password    NoteProperty string Password=Pa$$w0r
surname     NoteProperty string surname=williams
```

Using a collection as input (`ForEach-Object`)

- Step 1: Define a collection of property names (usually the name of the account).
  - You can either use it directly in the `ForEach` or assign the collection to a variable and use the variable.

```
$userCollection = Import-Csv .\userNames.csv
```

- Step 2: Using a `ForEach-Object` loop, iterate through the array one object at a time.

```
$userCollection | ForEach-Object {

    New-ADuser -name $_.name -givenName $_.GivenName -surName $_.SurName -AccountPassword (ConvertTo-SecureString $_.Password -AsPlainText -Force)

}
```

That looks gross. Let’s Splat it.

- “splatting” is the process of storing all property/values pairs in a hash table and calling that hash table when the command runs.
- Splatting allows use to use natural line continuators to clean up our code.
- It’s also “containerizes” our code making it easy to read, troubleshoot and modify.
- Making code “modular” is a great idea! You can reuse portions of code your happy with, elsewhere.

ForEach-Object splatted

Put the collection (whatever it may be) in the pipe 1

2 Pipe the collection to `forEach-Object` to start the iterative loop.

```
Import-Csv .\userNames.csv |
    ForEach-Object {

        $userCreationParams = @{
            Name            = $_.name
            GivenName       = $_.GivenName
            SurName         = $_.SurName
            AccountPassword = (ConvertTo-SecureString $_.Password -AsPlainText -Force)
        }

        New-Aduser @userCreationParams -Path "ou=pwshUsers,dc=geoffb,dc=local" -PassThru | Format-List name
    }
```

3 Initialize a variable with a `@{}` (hash table)

5 Assign each variable a value. This can be "hard coded", or it can be derived from your collection

4 Create a named item with the exact name of each parameter in your command

6 Call the Splat when you run the command. This will match each object in the hash table with the parameter that shares its name. (you can also hard code values and use Splatting together)

Bonus: `For`!

- `For` is not the best option for bulk creation. `For` is best used for counting in loops or incrementing numbers. We could still find a use for it, though.
- Example:
  - I want to create 10 new computer accounts called Computer-1 → Computer-10.
- Because the only thing changing in the name is the number, a `For` loop would be perfect here.

For (creating 10 similarly named computers)

Initialize @computerNum as 1 (our first computer will be Computer-1)

Set a condition for the loop to exit. If $computerNum is >= 11, exit the loop

Each time the loop iterates, increment $computerNum by 1

```
For ($computerNum = 1; $computerNum -lt 11; $computerNum++){

    New-AdComputer -name "Computer-$ComputerNum" -Description "This is computer $computerNum!" -PassThru |
        Select-Object SamAccountName

}
```

During the first loop $ComputerNum = 1; during the second loop $computerNum = 2; etc.